Universidade de Caxias do Sul
Área do Conhecimento de Ciências Exatas e Engenharias
CIC4004X - Programação Concorrente, Paralela e Distribuída
Professor: André Luis Martinotto

QUESTÃO 1) (1,0 PONTO) Na taxonomia de Flynn:

a) O que significa SIMD? Forneça uma breve descrição e um exemplo.
b) O que significa MIMD? Forneça uma breve descrição e um exemplo.
c) Os exemplos citados anteriormente podem ser classificados com um sistema distribuído? Por quê?

QUESTÃO 2) (1,0 PONTO) Indique para cada ferramenta abaixo:

- O tipo de memória predominante: memória compartilhada, memória distribuída e memória compartilhada em GPU
- Se ferramenta é mais adequada ao desenvolvimento de: aplicações paralelas, concorrentes ou distribuídas;

Justifique sua resposta.

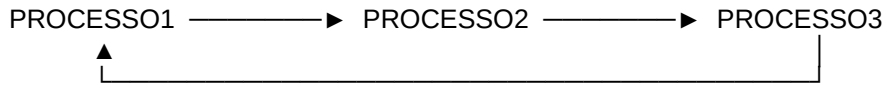
Ferramenta	Tipo de memória predominante	Tipo de aplicação (paralela/concorrente/distribuída)	Justificativa
OpenMP			
MPI			
Pthreads			
CUDA			
Sockets			

QUESTÃO 3) (1,0 PONTO) Considere os diferentes tipos de aplicações a seguir. Para cada uma, indique qual protocolo de transporte é mais adequado — TCP ou UDP — justificando brevemente sua escolha com base nas características da aplicação (como confiabilidade, tempo real, ou tolerância a perdas).

Situação	Protocolo mais adequado (TCP ou UDP)	Justifique brevemente
Chamadas de voz pela Internet (VoIP).		
Transferência de arquivos (ex.: FTP, SCP).		
Jogos online em tempo real (ex.: FPS).		
Atualizações de firmware para dispositivos em rede local via multicast.		
Acesso a páginas web (HTTP/HTTPS).		

QUESTÃO 4) Considere um sistema distribuído composto por três processos, denominados PROCESSO1, PROCESSO2 e PROCESSO3, executados em computadores distintos. O PROCESSO1 é responsável por iniciar a comunicação. Ele deve enviar uma mensagem para o PROCESSO2. Ao receber a mensagem, o PROCESSO2 deve repassá-la integralmente para o PROCESSO3. Por sua vez, ao receber a mensagem, o PROCESSO3 deve repassá-la novamente para o PROCESSO1.

Dessa forma, o fluxo da mensagem deve obedecer ao seguinte caminho:



Considere a seguinte configuração de rede:

Processo	Endereço IP	Porta
PROCESSO1	192.168.0.10	5001
PROCESSO2	192.168.0.20	5002
PROCESSO3	192.168.0.30	5003

Considere, ainda, que os processos serão inicializados na seguinte ordem:

PROCESSO3 → PROCESSO2 → PROCESSO1

a) (1,0 PONTO) Implemente, em pseudocódigo, os três processos utilizando sockets TCP/IP, de forma que a mensagem enviada pelo PROCESSO1 percorra o caminho:

PR0CESS01 → PR0CESS02 → PR0CESS03 → PR0CESS01

b) (1,0 PONTO) Reimplemente a mesma situação utilizando sockets UDP/IP. O fluxo da mensagem deverá permanecer:

PR0CESS01 → PR0CESS02 → PR0CESS03 → PR0CESS01

c) (1,0 PONTO) Considere agora que o PROCESSO1 precisa enviar a mesma mensagem diretamente para o PROCESSO2 e para o PROCESSO3. Para essa situação, qual protocolo de transporte (TCP ou UDP) você escolheria? Justifique sua resposta considerando as características dos dois protocolos.

Para o desenvolvimento dos pseudocódigos, considere as seguintes primitivas, baseadas na biblioteca de sockets da linguagem Python:

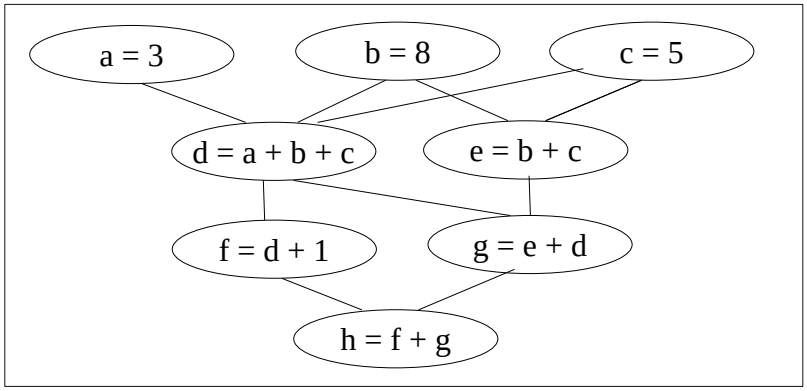
```
socket(AF_INET, SOCK_STREAM)
socket(AF_INET, SOCK_DGRAM)
bind(socket, (ip, porta))
listen(socket, quantidade)
connect(socket, (ip, porta))
novo_socket, endereco = accept(socket)
send(socket, dados)
sendto(socket, dados, (ip, porta))
dados = recv(socket, tamanho)
dados, endereco = recvfrom(socket, tamanho)
close(socket)
```

QUESTÃO 5) (1,0 PONTO) O MessagePack é um formato binário compacto que preserva estruturas de dados semelhantes às do JSON, o qual utiliza uma representação textual. Considerando essas características, compare MessagePack e JSON, destacando as principais vantagens de cada formato. Além disso, explique de que maneira a escolha entre eles pode influenciar o desempenho e o comportamento de aplicações distribuídas.

QUESTÃO 5) (2,0 PONTOS) Seja o seguinte trecho de programa:

```
a = 3
b = 8
c = 5
d = a + b + c
e = b + c
f = d + 1
g = e + d
h = f + g
```

O grafo abaixo, representa o grafo de precedência correspondente ao trecho acima que represente a concorrência máxima possível de ser obtida sem que os valores finais das variáveis difiram daqueles obtidos através da execução seqüencial.



Utilizando uma biblioteca de troca de mensagens baseada no modelo SPMD (um único programa operando sobre múltiplos dados), e considerando as primitivas apresentadas abaixo, desenvolva um programa que implemente o grafo da Figura 1. Cada operação deverá ser executada por um processo distinto, garantindo o correto atendimento das dependências entre as operações.

```
rank()
size()
send(<var>, <id>)
receive(<var>, <id>)
```

QUESTÃO 4) (1,0 PONTO) Um programa que realiza um processamento intensivo leva 100 segundos para ser executado em um único núcleo de CPU. Após a implementação de uma versão paralela, o mesmo programa é executado em 4 núcleos de CPU e leva 30 segundos para ser concluído.

a) Calcule o speedup obtido com a execução do programa em paralelo.

b) Calcule a eficiência do programa em relação ao número de núcleos utilizados.

Solução – Primeira Avaliação

QUESTÃO 1) Na taxonomia de Flynn:

a) SIMD (*Single Instruction, Multiple Data*): nesse tipo de arquitetura uma única instrução é executada sobre um conjunte dados. Em geral, esse tipo de arquitetura opera uma única instrução sobe um conjunto de elementos de um vetor ou matriz. Exemplos de arquiteturas: processadores vetoriais e placas de processamento gráfico (*GPUs*)

b) MIMD (*Multiple Instruction Multiple Data*): nesse tipo de arquitetura múltiplas instruções são executadas, e cada uma delas operando sobre dados independentes. Exemplo de arquiteturas: máquinas multiprocessadas ou multicores.

c) As arquiteturas listadas não são exemplo de sistema distribuídos, uma vez que essas apresentam uma memória compartilhada e um relógio global.

QUESTÃO 2)

Ferramenta	Tipo de memória predominante	Tipo de aplicação (paralela /concorrente/distribuída)	Justificativa
OpenMP	Memória compartilhada	Paralela	Utiliza múltiplas threads. É usado para paralelização em múltiplos núcles de processamento.
MPI	Memória distribuída	Paralela	É usado para paralelização em clusters de computadores
Pthreads	Memória compartilhada	Concorrente/Paralelas	É usado tanto para aplicações concorrentes quanto paralelas.
CUDA	Memória compartilhada em GPU	Paralela	Explora o paralelismo em GPUs
Sockets	Memória distribuída	Distribuída	São usados para aplicações distribuídas e sistemas cliente-servidor.

QUESTÃO 3)

Situação	Protocolo mais adequado (TCP ou UDP)	Justifique brevemente
Chamadas de voz pela Internet (VoIP).	UDP	Priorizar baixa latência e tempo real em vez de confiabilidade.
Transferência de arquivos (ex.: FTP, SCP).	TCP	Exige confiabilidade e entrega completa dos dados
Jogos online em tempo real (ex.: FPS).	UDP	Respostas rápidas e podem tolerar a perda de pacotes
Atualizações de firmware para dispositivos em rede local via multicast.	UDP	Permite enviar o mesmo pacote a vários dispositivos simultaneamente
Acesso a páginas web (HTTP/HTTPS).	TCP	Exige confiabilidade e entrega completa dos dados

QUESTÃO 4)

a)

PROCESSO 1

```
import socket

ip = 'localhost'
porta_recebe = 5001
porta_envia = 5002
envia = socket.socket(socket.AF_INET,
                      socket.SOCK_STREAM)
envia.connect((ip, porta_envia))
envia.send("Mensagem".encode())
s = socket.socket(socket.AF_INET,
                  socket.SOCK_STREAM)
s.bind((ip, porta_recebe))
s.listen(10)
recebe, cliente = s.accept()
msg = recebe.recv(1024).decode()
print('Recebi: %s' % msg)
s.close()
recebe.close()
envia.close()
```

PROCESSO 2

```
import socket

ip = 'localhost'
porta_recebe = 5002
porta_envia = 5003
s = socket.socket(socket.AF_INET,
                  socket.SOCK_STREAM)
s.bind((ip, porta_recebe))
s.listen(10)
recebe, cliente = s.accept()
msg = recebe.recv(1024).decode()
envia = socket.socket(socket.AF_INET,
                      socket.SOCK_STREAM)
envia.connect((ip, porta_envia))
envia.send(msg.encode())
s.close()
recebe.close()
envia.close()
```

```
import socket

ip = 'localhost'
porta_recebe = 5003
porta_envia = 5001

s = socket.socket(socket.AF_INET,
                  socket.SOCK_STREAM)
s.bind((ip, porta_recebe))
s.listen(10)
recebe, cliente = s.accept()
msg = recebe.recv(1024).decode()
envia = socket.socket(socket.AF_INET,
                      socket.SOCK_STREAM)
envia.connect((ip, porta_envia))
envia.send(msg.encode())
s.close()
recebe.close()
envia.close()
```

b)

PROCESSO 1

```
import socket

ip = 'localhost'
porta_recebe = 5000
porta_envia = 5001
envia = socket.socket(socket.AF_INET,
                      socket.SOCK_DGRAM)
envia.sendto('Mensagem'.encode(), (ip, porta_envia))
recebe = socket.socket(socket.AF_INET,
                       socket.SOCK_DGRAM)
recebe.bind((ip, porta_recebe))
dados, cliente = recebe.recvfrom(1024)
print('%s' % dados.decode())
envia.close()
recebe.close()
```

PROCESSO 2

```
import socket

ip = 'localhost'
porta_recebe = 5001
porta_envia = 5002
recebe = socket.socket(socket.AF_INET,
                       socket.SOCK_DGRAM)
recebe.bind((ip, porta_recebe))
dados, cliente = recebe.recvfrom(1024)
envia = socket.socket(socket.AF_INET,
                      socket.SOCK_DGRAM)
envia.sendto(dados, (ip, porta_envia))
envia.close()
recebe.close()
```

PROCESSO 3

```
import socket

ip = 'localhost'
porta_recebe = 5002
porta_envia = 5000

recebe = socket.socket(socket.AF_INET,
                       socket.SOCK_DGRAM)
recebe.bind((ip, porta_recebe))
dados, cliente = recebe.recvfrom(1024)
envia = socket.socket(socket.AF_INET,
                      socket.SOCK_DGRAM)
envia.sendto(dados, (ip, porta_envia))
envia.close()
recebe.close()
```

c) Protocolo UDP, pois ele permite enviar a mesma mensagem do PROCESSO1 para o PROCESSO2 e o PROCESSO3 sem precisar estabelecer uma conexão previamente.

QUESTÃO 5)

O MessagePack é um formato binário que, por ser mais compacto, reduz a quantidade de dados transmitidos e oferece uma conversão mais rápida durante a serialização e deserialização. Em contraste, o JSON utiliza um formato textual, que é mais legível e facilita a integração, mas exige uma maior transmissão de dados, resultando em um aumento no uso de rede e no tempo de transmissão. Além disso, o JSON demanda um tempo maior para a conversão, tanto na serialização quanto na deserialização.

QUESTÃO 6)

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char **argv){
	int id, a, b, c, d, e, f, g, h; MPI_Status s;

	MPI_Init(&argc, &argv);
	MPI_Comm_rank(MPI_COMM_WORLD, &id);

	switch ( id ){
		case 1:
			b = 8;
			MPI_Send(&b, 1, MPI_INT, 3, 100, MPI_COMM_WORLD);
			MPI_Send(&b, 1, MPI_INT, 4, 100, MPI_COMM_WORLD);
			break;
		case 2:
			c = 5;
			MPI_Send(&c, 1, MPI_INT, 3, 100, MPI_COMM_WORLD);
			MPI_Send(&c, 1, MPI_INT, 4, 100, MPI_COMM_WORLD);
			break;
		case 3:
			MPI_Recv(&a, 1, MPI_INT, 0, 100, MPI_COMM_WORLD, &s);
			MPI_Recv(&b, 1, MPI_INT, 1, 100, MPI_COMM_WORLD, &s);
			MPI_Recv(&c, 1, MPI_INT, 2, 100, MPI_COMM_WORLD, &s);
			d = a + b + c;
			MPI_Send(&d, 1, MPI_INT, 5, 100, MPI_COMM_WORLD);
			MPI_Send(&d, 1, MPI_INT, 6, 100, MPI_COMM_WORLD);
			break;
		case 4:
			MPI_Recv(&b, 1, MPI_INT, 1, 100, MPI_COMM_WORLD, &s);
			MPI_Recv(&c, 1, MPI_INT, 2, 100, MPI_COMM_WORLD, &s);
			e = b + c;
			MPI_Send(&e, 1, MPI_INT, 6, 100, MPI_COMM_WORLD);
			break;
		case 5:
			MPI_Recv(&d, 1, MPI_INT, 3, 100, MPI_COMM_WORLD, &s);
			f = d + 1;
			MPI_Send(&f, 1, MPI_INT, 7, 100, MPI_COMM_WORLD);
			break;
		case 6:
			MPI_Recv(&d, 1, MPI_INT, 3, 100, MPI_COMM_WORLD, &s);
			MPI_Recv(&e, 1, MPI_INT, 4, 100, MPI_COMM_WORLD, &s);
			g = d + e;
			MPI_Send(&g, 1, MPI_INT, 7, 100, MPI_COMM_WORLD);
			break;
		case 7:
			MPI_Recv(&f, 1, MPI_INT, 5, 100, MPI_COMM_WORLD, &s);
			MPI_Recv(&g, 1, MPI_INT, 6, 100, MPI_COMM_WORLD, &s);
			h = f + g;
			printf("g: %d\n", h);
			break;
	}
	MPI_Finalize();
}
```

Questão 7)

a) O speedup é uma métrica que considerada quantas vezes o programa executado em paralelo é mais rápido do que sua versão sequencial. Esse é a razão entre o tempo de execução de uma tarefa sequencial e o tempo de execução da mesma tarefa em um sistema paralelo.

$$S = \frac{T_s}{T_p} = \frac{100}{30} = 3{,}33$$

b) A eficiência indica quão bem os recursos estão sendo aproveitados em relação ao ideal. Essa é a razão entre o speedup e o número de processadores utilizados,

$$E = \frac{S}{N_{proc}} = \frac{3{,}33}{4} = 0{,}8333$$